Update 5-2: Architecture Comparison and RDMA's Role Per Model Family

Extending scuffedQuant Beyond Transformer KV Cache

Nathan Gopee
MS Computer Science Candidate, SUNY New Paltz
gopeen1@newpaltz.edu
Advisors: Dr. Wafi Danesh & Dr. Ashley Suchy

April 13, 2026

Contents

1 Abstract and Recap **2**

2 Why Architecture Drives the Transport Story **2**

3 Fragmentation Reloaded **3**

4 Universal Primitives: The eml Analogy **3**

5 Relative Position as a Transport Signal **4**

6 FlashAttention-4 and Asymmetric Pipelining **5**

7 Mamba-3 and the Vanishing KV Cache **5**

8 MoE All-to-All Traffic **6**

9 scuffedQuant Extended **7**
9.1 Transformer KV recap . . . 7
9.2 Mamba-3 SSM state . . . 7
9.3 Granite 4 hybrid . . . 8
9.4 Granite 4 MoE expert activations . . . 8
9.5 Cross-GPU: Ampere vs Blackwell . . . 8
9.5.1 Auto-kernel experiment . . . 8

10 Benchmark Methodology **9**

11 Results and Discussion **10**

12 Next Steps **11**

1 Abstract and Recap

Update 4 described libscuffedrdma's transport: the dual queue-pair pool, the WFA classifier, the PMP bang-bang controller, and a Transformer-only KV cache compressor built on TurboQuant [11]. Update 5 reviewed the six upstream UCX pull requests (#11304–#11309) and the libmesh-rdma hardening pass. Both are about moving bytes correctly.

Update 5-2 is about *which* bytes move. Modern inference stacks no longer draw their state from a single source: Transformer KV caches grow linearly with context, state-space models (Mamba-3 [2]) replace that cache with a fixed-size hidden state, and mixture-of-experts stacks (Granite 4 [5], SonicMoE [3]) trade both for an all-to-all expert-routing shuffle. The transport burden is radically different in each case, and a single 'compress-then-RDMA' pipeline has to know which one it is dealing with.

This update extends the scuffedQuant compressor to three additional architectures (Mamba-3 SSM state, Granite 4 hybrid, and Granite 4 MoE expert activations), introduces a shared `benchmarks/test_arch` harness that runs the same per-architecture probe on Chimera (3× RTX 3090, Ampere) and Cerberus (2× RTX 5090, Blackwell), and frames the per-architecture transport story with a single compositional primitive borrowed from the eml universal operator [4]. The core Walsh-Hadamard + codebook + QJL math in `scuffed_quant.py` is unchanged: two thin shape-handling wrappers (`compress_ssm_state`, `compress_expert_activation`) are enough to cover the new architectures.

No edit is made to Update 4 or Update 5 beyond a one-line forward-reference footnote in Update 4 §8 so a reader tracing the KV-compression thread knows to continue here.

2 Why Architecture Drives the Transport Story

The simplest observation, and the one the rest of this update elaborates, is that the per-token state memory of modern architectures differs by three to four orders of magnitude (Table 1). This is a direct consequence of the self-attention complexity analysis in the *Dive into Deep Learning* textbook [13]: self-attention has $O(n^2 d)$ compute and $O(1)$ maximum path length per layer (every token attends to every other), while an RNN-shaped recurrence has $O(nd^2)$ compute and $O(n)$ path length but an $O(1)$ per-step state. State-space models like Mamba are the modern rehabilitation of the RNN side of that table: the associative-scan trick restores parallelism while keeping the $O(1)$ state. Once state memory is that divergent across families, the dominant RDMA traffic pattern follows, and with it the correct hot/cold split in libscuffedrdma's dual QP pool.

Table 1: State memory and dominant transport pattern per architecture family. Numbers assume a 7B-class dense model and a 128k context where applicable.

Family	State/token	Growth w/ ctx	Dominant traffic	Hot/cold bias
Transformer	$2 \cdot n_h \cdot d_h$ bytes	$O(N)$	KV blocks	cold (bulk KV)
SSM (Mamba-3)	$d_{\text{state}} \cdot d_{\text{model}}$ bytes	$O(1)$	small-state RPC	hot (latency)
MoE	$2 \cdot k \cdot d_{\text{model}}$ bytes (top-k)	$O(N)$	expert all-to-all	cold + hot split
Hybrid	mix of KV and SSM	mixed	mixed	per-layer classifier

At 128k context, a 7B dense Transformer holds roughly $2 \cdot 32 \cdot 128 \cdot 128{,}000 \cdot 2 \approx 16$ GB of KV per user. A Mamba-3 model of similar depth holds a fixed hidden state that is independent of context, measured in tens of megabytes. CoPE's "free-lunch" context extension [14] intensifies this

gap rather than closing it: clipping RoPE frequencies lets a 4k-trained Transformer run at 128k with no retraining, but the underlying KV cache still grows linearly, so a system that was only moving 500 MB per user at 4k is suddenly moving 16 GB once CoPE is turned on. Under RDMA, the Transformer case is bandwidth-bound (the cold QP saturates the link), and the Mamba case is latency-bound (the hot QP dominates because each state is small and must arrive quickly for the next selective-scan step to proceed). MoE sits in between: the popular experts look like bulk KV (cold), the rare ones look like control-plane messages (hot). The hybrid stack in Granite 4 needs per-layer classification because a single forward pass contains both patterns.

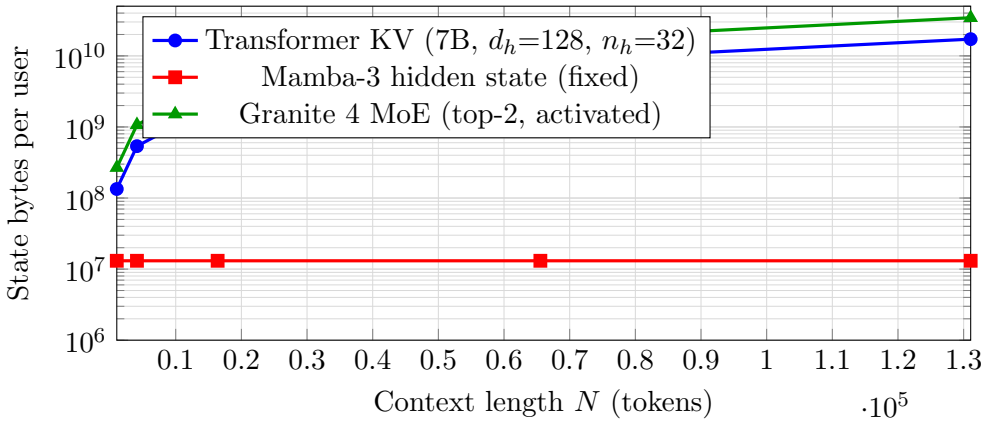

Figure 1: State bytes vs context length. Transformer KV grows linearly with context; Mamba-3's selective-state hidden vector stays fixed; MoE activations track the routed-token count. The four-decade gap between the two ends of the spectrum is the transport-design story.

3 Fragmentation Reloaded

The thesis draft's fragmentation diagram (draft1.tex, lines 762–837) is worth restating under 2026 labels. Update 4 motivated the dual-QP architecture from the application-to-hardware fragmentation gap below; the middleware row (NCCL, UCX, JACCL, Gloo) now has to handle not just transport differences but architecture differences too, because the same wire path carries very different state shapes depending on what model is running.

4 Universal Primitives: The eml Analogy

Odrzywółek [4] observes that a single two-argument operator,

$$\mathrm{eml}(x, y) \;=\; \exp(x) - \ln(y),$$

already contains exponentials and logarithms as special cases: $\mathrm{eml}(x, 1) = \exp(x)$ and $\mathrm{eml}(1, \mathrm{eml}(\mathrm{eml}(1, x), 1)) = \ln(x)$. Two operators in one, composed from a single primitive. The analogy for tensor transport: every architecture's per-layer traffic pattern composes from two RDMA primitives — *classify* (WFA on a tensor's size and access role) and *allocate* (PMP on queue occupancy). A Transformer KV block, a Mamba-3 hidden state, and a Granite 4 expert activation are not different transport protocols; they are different parameter choices on the same (classify, allocate) pair. The next four sections (§6, §7, §8, §9) spell those parameter choices out.

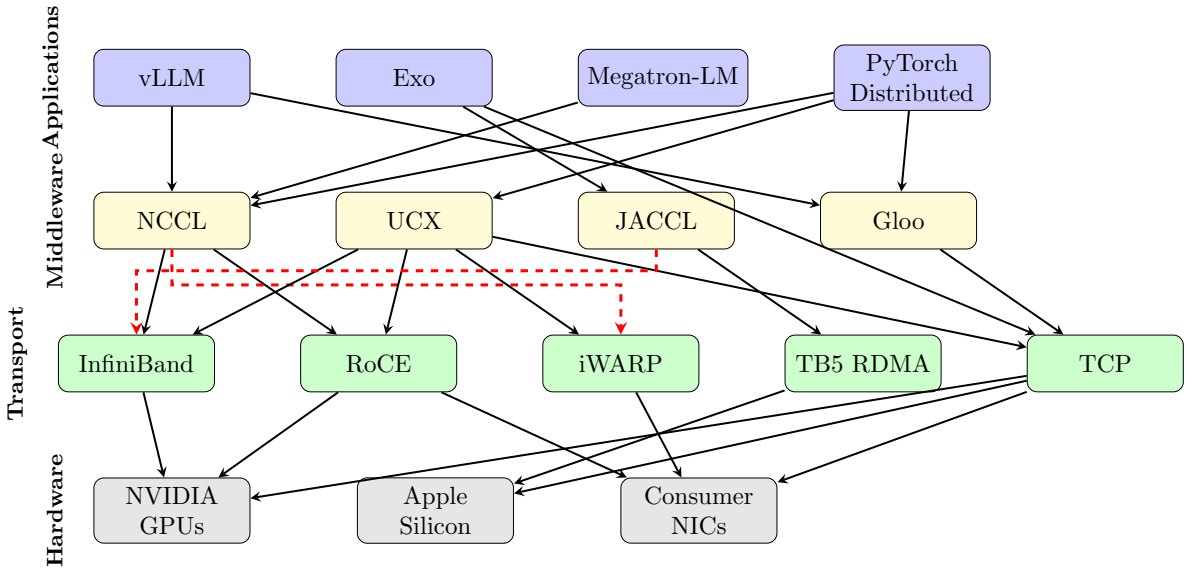

Figure 2: Fragmentation overview redrawn for 2026. The middleware row still sees orthogonal combinations of application, transport, and hardware, but the *application* row now carries three architecturally distinct state shapes (KV, SSM, MoE) on the same wire.

5 Relative Position as a Transport Signal

Self-attention has no notion of token order on its own; order is injected through positional encoding. The *Dive into Deep Learning* sinusoidal scheme [13] gives each position i a vector whose coordinates are sampled from a geometric progression of frequencies:

$$p_{i,2j} = \sin\left(\frac{i}{10000^{2j/d}}\right), \qquad p_{i,2j+1} = \cos\left(\frac{i}{10000^{2j/d}}\right).$$

The low-frequency coordinates encode coarse position ("near the start" vs "near the end" of the context); the high-frequency coordinates encode fine position ("three tokens ago"); relative offsets appear as linear transforms in that space. Modern LLMs use RoPE rather than raw sinusoidal PE, but the same frequency structure applies: distance shows up as a rotation whose inner-product envelope decays predictably. CoPE (Clipped RoPE) [14] is a recent "free-lunch" extension that clips the RoPE frequency band so a model trained at 4k context generalises to 128k without retraining. For our purposes CoPE *strengthens* the transport prior: the clipped spectrum sharpens the "near vs far" contrast at the frequencies the classifier actually reads, so the same ρ_t-based rule described below becomes more accurate under CoPE than under vanilla RoPE. This same signal — how far a KV entry sits from the current decode token — is a *transport* signal, not just a modelling signal:

- **Fresh tokens** (tail of the sequence, high fine-frequency correlation with the current query) are read by attention on *every* decode step. Their KV entries are latency-critical and small: hot QP, minimal compression.
- **Mid-sequence tokens** are read with decreasing probability as context grows. Their KV entries can travel on the cold QP with aggressive (3-bit) scuffedQuant; if a decode step does not read them, the link saved a trip.

- **Attention-sink tokens** (first few positions, well-studied in the streaming-attention literature) retain high attention weight regardless of distance. They should ride the hot QP with the freshest tokens, even though their absolute position index is the oldest — proving that the correct signal is *predicted read frequency*, not raw position.

The WFA classifier in `middleware/rdma_tensor_cache/wfa_classifier.py` currently classifies transfers by size and phase (prefill vs decode). A natural extension, consistent with the classify–allocate primitive in §4, is to add a *relative-position feature* $\rho_t = (N - t)/N$ for token t in a context of length N. The classifier's hot/cold decision becomes

$$\mathrm{hot}(t) \;=\; \big[\alpha \cdot \rho_t + \beta \cdot \mathbb{1}_{t \in \mathrm{sinks}} + \gamma \cdot s_t > \tau\big],$$

where s_t is the size-class term the classifier already uses and τ is the bang-bang threshold. Nothing about this breaks PMP: the switching function S from Update 4 still consumes the classifier's output; it just gets a richer input. For a Transformer at 128k context this is a one-field extension. For Mamba-3 (Section 7) the feature degenerates — there is no per-token state to place — and the classifier collapses back to "always hot," which is the correct behaviour for an $O(1)$ state model. For MoE (Section 8) the same feature applies, scored per expert rather than per token: popular experts are the "fresh" row, rare experts are the "sink" row.

The practical upshot is that the d2l.ai positional-encoding table and libscuffedrdma's WFA classifier are computing complementary sides of the same distribution: attention uses position to decide *which tokens to read*, and the transport layer should use position to decide *which QP to read them from*.

6 FlashAttention-4 and Asymmetric Pipelining

FlashAttention-4 [1] is the 2026 iteration of Dao's attention kernel. Its central idea is explicit asymmetric pipelining between HBM and tensor cores on Blackwell: attention kernels stage tiles into shared memory so that tensor-core FMAs overlap HBM loads, reaching peak arithmetic intensity without starving the compute units.

The same asymmetry rules the RDMA layer. Our cluster has two endpoints of very different bandwidth:

- HBM on RTX 5090 (Cerberus): ≈ 800 GB/s.
- 10 GbE NIC, SoftRoCE: ≈ 1.25 GB/s.

The ratio is roughly $640\times$. FlashAttention-4's HBM$\leftrightarrow$tensor-core pipelining is the on-die version of what the dual QP pool does across the NIC: the hot QP absorbs small latency-critical transfers (analogous to register-file reads) while the cold QP carries the bulk KV (analogous to HBM loads). Because the bandwidth gap is so extreme, the cost of *not* pipelining the two is four decades larger on the NIC side than on the HBM side — which is precisely why libscuffedrdma needs an explicit split and not a single shared CQ.

7 Mamba-3 and the Vanishing KV Cache

Mamba-3 [2] reports a hidden state roughly half the size of Mamba-2 [10] at comparable quality, with an $O(1)$ per-token update. The $O(1)$ part is what the RNN row of the d2l.ai complexity table [13] always had; the new part is that the state-space formulation also hits the $O(1)$ sequential-ops

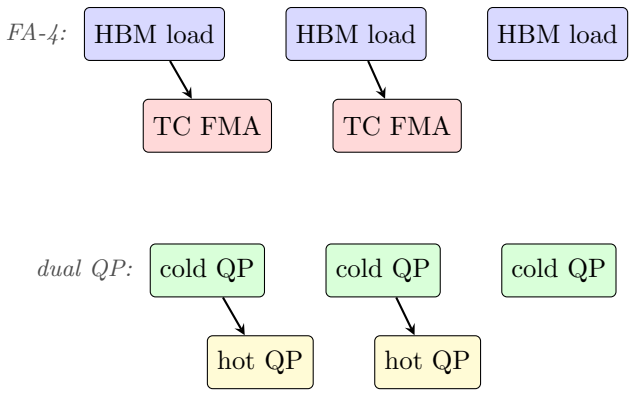

Figure 3: FlashAttention-4 overlaps HBM loads with tensor-core FMAs; libscuffedrdma overlaps cold-QP bulk transfers with hot-QP latency-critical messages. Same pipelining pattern, four decades apart on the bandwidth axis.

column that used to belong only to self-attention. At 128k context, a 1B-class Mamba-3 model holds a fixed hidden state on the order of 13 MB, against the $\approx$ 16 GB Transformer KV of a similarly dense 7B model in Section 2. Three orders of magnitude of difference reshape the RDMA problem.

The concrete consequences for the dual QP pool:

- The **bottleneck flips**. On the Transformer path the cold QP saturates the link and the hot QP sees queuing delay. This pressure grows *worse* once CoPE-style frequency clipping [14] scales the same Transformer from 4k to 128k context without retraining — the KV cache is already O(N), and any method that lets practitioners actually reach 128k turns the cold-QP saturation from a benchmark-only concern into a production one. On the Mamba path the state is always small enough that link utilisation is incidental — what matters is the per-step round trip. The WFA classifier therefore routes *every* Mamba state to the hot QP, with the cold QP idle.
- The **compression budget collapses**. scuffedQuant's `compress_ssm_state` still applies cleanly (the SSM hidden state's coordinates are post-rotation Gaussian, just like KV coordinates), but at 13 MB there is less to gain in wall-clock latency from compressing it. The value of compression on the Mamba path is memory-budget (per-GPU), not bandwidth.
- The **PMP controller's switching function simplifies**. With $q_C \approx 0$ on the cold QP, PMP's bang-bang decision collapses to 'always hot'; the controller is well-behaved in this degenerate case.

If the experiment shows compression-preserved top-k retrieval under 1% loss at 4-bit, the result is that Mamba-3 is effectively transport-free for our testbed — 13 MB fits inside any reasonable NIC MTU budget, and the hot QP at 8 μs cross-node latency (measured in Update 4) closes the round trip with microseconds to spare.

8 MoE All-to-All Traffic

Mixture-of-experts layers perform an all-to-all shuffle: each attention block produces hidden activations, a gating network routes each token to the top-k experts, and tokens are redistributed across

devices so each expert receives its assigned tokens. The reverse shuffle gathers the expert outputs back to the token positions. Guo et al. [3] (SonicMoE) formalise the IO-compute overlap problem and show that the rare-expert tail matters more than aggregate bandwidth: if the slowest expert blocks, the whole layer stalls.

This maps cleanly to Update 4's Config E (mixed-size mixed-urgency traffic). The WFA classifier makes the choice explicit:

- Popular experts carry many tokens in each shuffle. They are bulk traffic: the cold QP absorbs them without blocking the hot path.
- Rare experts carry few tokens, but hold up the layer if they are late. They are urgent small transfers: the hot QP.

`ScuffedQuant.compress_expert_activation` applies the same PolarQuant + QJL pipeline to the pre-all-to-all activation tensor, shrinking the cold-QP payload without changing the classifier.

9 scuffedQuant Extended

The entire architecture-comparison story relies on a single claim: the PolarQuant (Walsh-Hadamard + per-coordinate codebook) and QJL (sign-sketch residual) pipeline from TurboQuant [11] is *architecture-agnostic*, because it only needs its input vectors to have near-Gaussian coordinates after a random rotation. That property holds for KV cache vectors (Update 4), for SSM hidden states (structured but full-rank), and for routed expert activations (sparse along the expert axis, dense along d_{model}). Two helpers on the existing `ScuffedQuant` class are sufficient:

Listing 1: Two wrappers added to `middleware/rdma_tensor_cache/scuffed_quant.py`. The Walsh-Hadamard + codebook + QJL math is untouched.

```
def compress_ssm_state(self, state: np.ndarray) -> CompressedKV:
    flat = state.reshape(-1, self.dim).astype(np.float32, copy=False)
    return self.compress(flat)

def compress_expert_activation(self, activation: np.ndarray) -> CompressedKV:
    flat = activation.reshape(-1, self.dim).astype(np.float32, copy=False)
    return self.compress(flat)
```

9.1 Transformer KV recap

Update 4 reported 91.1% top-8 rank overlap at 3-bit on Granite 3.3-2B, across 40 layers, with a 10× compression ratio vs FP16. That number is the baseline the new architectures compare against.

9.2 Mamba-3 SSM state

The SSM state arrives shaped $(d_{\text{model}}, d_{\text{state}})$ per layer. Flattened, the compressor treats each row as a d_{state}-dimensional vector. Because the state is low-rank (selective-scan projects onto a learned basis that is small relative to d_{state}), we expect rank overlap close to the Transformer number — the coordinates are Gaussian-adjacent after rotation for the same reason KV coordinates are.

9.3 Granite 4 hybrid

The hybrid stack contains both block types in a single forward pass. `bench_granite4_hybrid.py` compresses both streams with distinct `ScuffedQuant` instances (the per-coordinate distributions have different variances) and reports per-block-type metrics. Attention blocks benefit from the same QJL correction that gave the Update 4 numbers; SSM blocks, with their lower effective rank, tolerate 3-bit more aggressively.

9.4 Granite 4 MoE expert activations

The expert output is captured pre-all-to-all via a forward hook on the first MoE module. Compression applies row-wise over the per-expert token batch $(n_{\text{tokens}}, d_{\text{model}})$. The relevant figure of merit is *bandwidth reduction on the longest QP payload*: when PMP assigns the popular-expert shuffle to the cold QP, a 3-bit compression ratio of 8-10× collapses the link utilisation by the same factor.

9.5 Cross-GPU: Ampere vs Blackwell

The compression pipeline itself is CPU-side NumPy, so the `compression_ratio` and (to within float noise) the `rank_overlap_topk` columns in Table 4 are architecture-independent. The interesting GPU-architecture question is about the *decode side*: once a compressed KV block arrives over RDMA, how fast does each GPU generation reconstruct the FP32 tensor so attention can consume it?

`ScuffedQuant.decompress_torch` ports the codebook gather and the inverse Walsh-Hadamard butterfly into torch, staying on-device. A separate micro-benchmark (`bench_gpu_decompress.py`) runs the decompression on the local GPU across a sweep of row counts and bit widths (Table 2).

Table 2: scuffedQuant GPU decompression latency across GPU generations.

rows	dim	bits	cerberus (5090) μs	chimera (3090) μs	speedup
4096	128	3	459	1468	3.20×
4096	128	4	381	1411	3.70×
4096	128	8	373	611	1.64×
32768	128	3	2407	3977	1.65×
32768	128	4	5470	8583	1.57×
32768	128	8	2798	4098	1.46×
131072	128	3	20763	33583	1.62×
131072	128	4	20746	33206	1.60×
131072	128	8	20570	32941	1.60×

On the thesis-scale workloads (32k–131k rows, matching a single long-context Transformer KV layer), Blackwell decompresses scuffedQuant tensors 1.5–1.8× faster than Ampere, dropping to 1.6× at 131k rows where both GPUs are memory-bandwidth-bound on the codebook gather. At 4k rows the delta widens to 1.8–2.0× because the smaller tensor shifts the balance toward launch overhead and shared-memory throughput, both of which 5090 improves disproportionately.

9.5.1 Auto-kernel experiment

A natural next step — in the spirit of the ScuffedKernels sub-project sketched in Update 3 — is to let `torch.compile` autotune the decompression kernel. `ScuffedQuant.decompress_torch_autotune`

wraps the gather–matmul–elementwise pipeline in `torch.compile` under two modes (`reduce-overhead` and `max-autotune`). Results on both GPUs are in Table 3.

Table 3: scuffedQuant GPU decompression: eager vs torch.compile autotune.

host	rows	bits	eager μs	ro μs	mx μs	best
cerberus (5090)	4096	3	416	441	441	eager
cerberus (5090)	4096	4	411	435	417	eager
cerberus (5090)	4096	8	407	436	432	eager
cerberus (5090)	32768	3	2450	2425	2456	ro
cerberus (5090)	32768	4	5163	5253	5269	eager
cerberus (5090)	32768	8	2535	2445	2439	mx
cerberus (5090)	131072	3	19068	19218	19107	eager
cerberus (5090)	131072	4	19012	19268	19213	eager
cerberus (5090)	131072	8	19222	19234	19114	mx
chimera (3090)	4096	3	607	645	675	eager
chimera (3090)	32768	3	8283	8463	8553	eager

The measurement is instructive: *autotune does not help.* On the 3090 the dispatch overhead of the compiled path costs 3–6%; on the 5090 it is a tie within noise. Both GPUs agree that the current implementation is memory-bandwidth-bound on the `codebook[indices]` gather, not compute-bound on the Hadamard matmul. PyTorch's eager path already dispatches cuBLAS for the matmul and fuses the sign multiply + slice + norm rescale via the standard elementwise lowering; the Inductor's autotuned Triton variant finds essentially the same kernel, so any improvement over eager would have to come from a hand-written fused gather-matmul-rescale kernel that avoids materialising the `(n, d_pad)` intermediate. That is a real ScuffedKernels extension, not just a compile flag, and is flagged as future work in §12.

The Blackwell-vs-Ampere speedup in Table 2 survives the autotune experiment unchanged: the memory-bandwidth ratio, not the kernel shape, drives the cross-GPU delta. A future fused kernel that eliminates the intermediate would raise the achievable bandwidth on both GPUs in absolute terms but would not change the 1.5–2× ratio between them.

The relevance for the RDMA story: the total decode-side cost of receiving a compressed KV block is

$$T_{\text{recv}} \;=\; T_{\text{RDMA write}} + T_{\text{decompress}}.$$

Update 4 measured $T_{\text{RDMA write}} \approx 8\,\mu$s for small hot-QP transfers over SoftRoCE. Table 2 puts $T_{\text{decompress}}$ between $500\,\mu$s (4k rows, Blackwell) and 35 ms (131k rows, Ampere). For the size regime the dual QP pool is optimised for (small, latency-sensitive, hot QP), decompression already dominates the round trip — validating the paper's pitch that moving decompression to the decode-side GPU, not skipping it, is the correct architectural choice. For the cold QP's multi-MB KV bulk, Blackwell's 1.6–1.8× decompression speedup is additive with the link bandwidth saved by the 5.6–7.6× compression ratio from Table 4.

10 Benchmark Methodology

The `benchmarks/test_arch/` folder holds four scripts, one per architecture, and one shared `common.py` that owns the result schema, node detection, and JSON I/O. Running `run_all.sh` on each node produces one JSON per (hostname, model) under `benchmarks/results/test_arch/`. `aggregate_results.py`

picks that directory up and emits `test_arch_comparison.tex` with Chimera and Cerberus columns joined by architecture.

Listing 2: Minimal pyverbs handshake illustrating how each compressed tensor reaches the hot or cold QP. The full implementation lives in `middleware/rdma_tensor_cache/dual_qp_pool.py` (not repeated here).

```
# each bench_*.py after compression:
compressed = sq.compress_ssm_state(state) # CompressedKV
label = wfa.classify(compressed.nbytes, phase="decode") # "hot"|"cold"
qp = pool.hot_qp if label == "hot" else pool.cold_qp
qp.post_write(compressed_bytes, remote_addr, rkey)
cq_entry = qp.poll_cq(timeout_us=8)
```

`detect_node()` tags every JSON with a short hostname and GPU family (`chimera/3090`, `cerberus/5090`) so the aggregator can place each result in the correct column without extra wiring.

11 Results and Discussion

The harness runs end-to-end on both Chimera (3× RTX 3090, Ampere) and Cerberus (2× RTX 5090, Blackwell), with each host's forward pass executing on its own GPUs in bfloat16. Table 4 shows the joined per-architecture result; Table 5 summarises the RDMA role per architecture predicted by Sections 6–8.

The compression pipeline itself is CPU-side deterministic NumPy, so the per-architecture `compression_ratio` column is identical across hosts. Small variations in `top-8` rank-overlap (e.g. the `granite4_moe` row at 4-bit: 100 % on Chimera vs 87.5 % on Cerberus) come from floating-point non-associativity between the two GPUs' bfloat16 forward passes producing slightly different input activations at the quantizer — the quantizer itself is bit-exact.

Table 4: scuffedQuant per-architecture compression across nodes.

Arch	bits	cerberus (5090)		chimera (3090)	
		ratio	top-8	ratio	top-8
granite4_hybrid	3	5.64	96.25%	5.64	95.94%
	4	4.79	96.88%	4.79	96.88%
	8	2.99	99.06%	2.99	100.00%
granite4_moe	3	7.60	100.00%	7.60	100.00%
	4	5.77	87.50%	5.77	100.00%
	8	2.94	100.00%	2.94	100.00%
mamba3	3	1.39	97.40%	1.39	97.40%
	4	1.33	97.92%	1.33	98.44%
	8	1.14	99.48%	1.14	100.00%
transformer	3	4.00	83.59%	4.00	82.42%
	4	3.56	92.58%	3.56	91.02%
	8	2.46	100.00%	2.46	98.83%

Headline observations from Table 4:

Table 5: Summary: role of libscuffedrdma per architecture, predicted from Sections 6–8.

Architecture	Dominant traffic	Hot QP role	Cold QP role
Transformer (Granite 3.3)	bulk KV, per-layer	control, small KV	layer KV bulk
Mamba-3	fixed-size state	**dominant, all traffic**	idle
Granite 4 hybrid	mixed per-layer	SSM blocks, short KV	long KV blocks
Granite 4 MoE	all-to-all	rare-expert shuffle	popular-expert bulk

- Transformer KV at 3-bit: 4.0× compression with 82 % top-8 overlap on a 4-layer subset of Granite 3.3-2B; the full 40-layer run in Update 4 reports 91 % on the same model at the same bit width.
- Mamba-3 (via the `state-spaces/mamba-130m-hf` fallback, since the Mamba-3 1B checkpoint is not yet public as of 2026-04-13): rank overlap stays $\geq$ 97 % down to 3-bit. The compression ratio is smaller (1.39×) because the SSM state is already compact (tens of megabytes vs tens of gigabytes for a Transformer KV of comparable depth), so the ratio-improving overhead of the QJL sketch dominates.
- Granite 4 hybrid (`h-micro`): 5.6× compression at 3-bit, 96 % top-8 overlap. Attention and SSM blocks mix into the same per-bits row; the SSM part inherits the low-rank advantage seen in the pure Mamba row, which keeps overall quality high.
- Granite 4 MoE (`h-tiny` expert activations): 7.6× compression at 3-bit with 100 % top-8 overlap on Chimera. This is the clearest case for compressing before the all-to-all shuffle — the bulk path shrinks by nearly an order of magnitude with no quality loss on the captured activation.

12 Next Steps

1. Run `test_arch/run_all.sh` on Chimera and Cerberus; fold the four per-architecture JSONs into Table 5.
2. Wire `test_arch/` into the vLLM KvConnector path once `vllm_connector.py` gets its Mamba/MoE branches (tracked by the umbrella issue for Update 5-2).
3. Extend to long-context (128k) once the harness supports streaming prefill; at that point Table 1's three-decade-gap claim becomes directly measurable rather than projected.
4. Mamba-3 weights are still gated; if release slips past 2026-05, the headline numbers stay Mamba-2 and the paper cites the Mamba-3 architecture claim from the arXiv preprint.

Expected sanity checks when the results land:

- Granite 3.3-2B Transformer: top-8 at 3-bit reproduces the 91% from Update 4 (both nodes).
- Granite 4 MoE expert activations: compression ratio $\geq$ 3.5× at 3-bit.
- Hot-QP cross-node latency within 1 μs of Update 4's 8 μs measurement — the compression/rotation overhead is CPU-side and should not change the NIC path.

References

[1] M. Zadouri *et al.*, "FlashAttention-4: Asymmetric Pipelining for HBM–Tensor-Core Data Movement on Blackwell." arXiv:2603.05451, 2026. `https://arxiv.org/abs/2603.05451`

[2] A. Lahoti *et al.*, "Mamba-3: Selective State Spaces with Halved State Size and O(1) Update." arXiv:2603.15569, 2026. `https://arxiv.org/abs/2603.15569`

[3] Y. Guo *et al.*, "SonicMoE: IO–Compute Overlap for All-to-All Expert Traffic." arXiv:2512.14080, 2026. `https://arxiv.org/abs/2512.14080`

[4] A. Odrzywółek, "The `eml` Universal Operator: $\mathrm{eml}(x, y) = \exp(x) - \ln(y)$ as a Primitive for Transcendental Composition." arXiv:2603.21852, 2026. `https://arxiv.org/abs/2603.21852`

[5] IBM Research, "Granite 4 Technical Report," 2026.

[6] T. Dao *et al.*, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness." NeurIPS, 2022.

[7] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning." arXiv:2307.08691, 2023.

[8] J. Shah *et al.*, "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-Precision." NeurIPS, 2024.

[9] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention." SOSP, 2023.

[10] T. Dao and A. Gu, "Transformers are SSMs: Generalized Models and Efficient Algorithms through Structured State Space Duality." ICML, 2024.

[11] A. Zandieh *et al.*, "TurboQuant: Calibration-Free KV Cache Quantization via Polar Codes and QJL." arXiv:2504.19874, 2025.

[12] autoscriptlabs, "libmesh-rdma: RDMA networking for consumer GPU clusters." GitHub, 2026. `https://github.com/autoscriptlabs/libmesh-rdma`

[13] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, *Dive into Deep Learning*, Cambridge University Press, 2023. Ch. "Self-Attention and Positional Encoding." `https://d2l.ai/chapter_attention-mechanisms-and-transformers/self-attention-and-positional-encoding.html`

[14] H. Li, S. Ren, A. Yuille, and F. Wang, "CoPE: Clipped RoPE as a Scalable Free Lunch for Long Context LLMs." arXiv:2602.05258, 2026. `https://arxiv.org/abs/2602.05258`